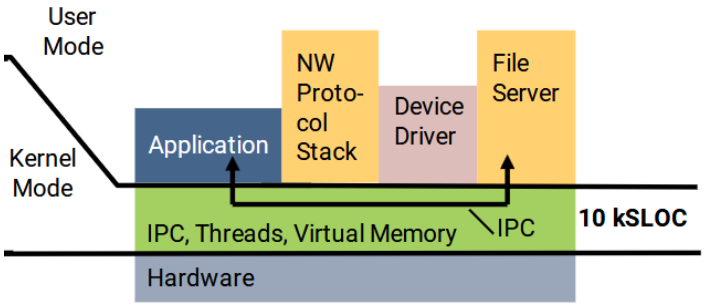
The SeL4 Microkernel - An Overview

Max Galetskiy

1. Overview



SeL4 is an OS microkernel. It's main features are:

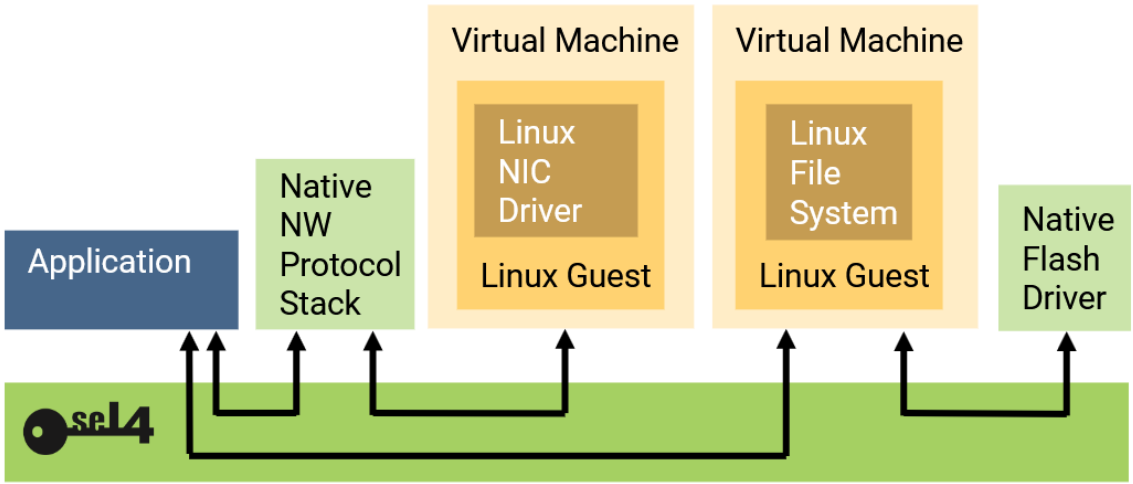
- small codebase ($\sim$ 10K LOC vs. Linux' 20 Mil. LOC)
- thus, small trusted computing base (TCB)[1]
- extremely low-level API
- can be used on it's own or as a hypervisor
- formal verification (correctness + security)

[1] subset of an overall system that must be trusted to operate correctly for the system to be secure

1.1. SeL4 as a Hypervisor

Why use SeL4 as a Hypervisor?

Example: Two services (networking, storage) implemented via Linux VMs



→ Untrusted VM does not have Kernel privileges

→ Untrusted VM does not interact with application

→ Communication only via SeL4-provided channels

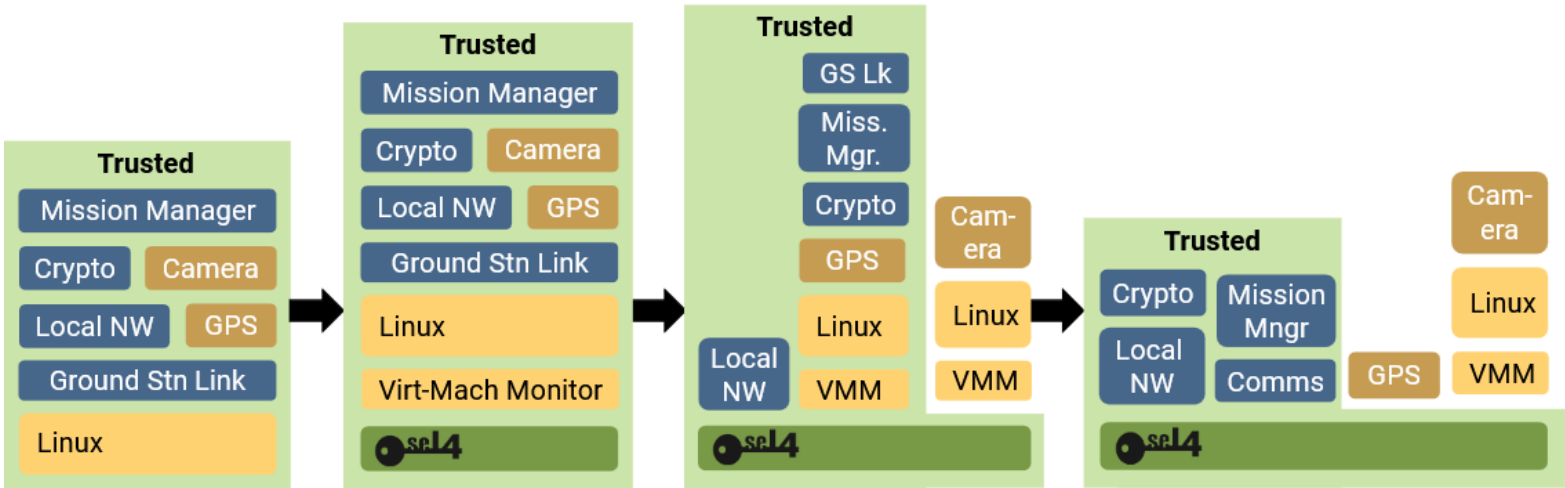
→ Features of Linux, security of SeL4

1.2. SeL4 in real Systems

Real-world systems rely significantly on legacy components that are expensive to port to SeL4.

Thus, the typical approach is **incremental cyber-retrofit**, i.e. push everything into a VM, then gradually separate and port components to SeL4.

Example: Boeing ULB mission computer:



2. Components

SeL4 provides these main components:

- **Capabilites** for access control
- **Threads** as the main execution model
- **Inter-Process Communication**
- **Virtual Address Spaces** for memory managment
- **Device Primitives** to communicate with I/O[2]
- Features to support Mixed-Criticality Systems specifically

[2] but drivers have to be made in user space

2.1. Capabilities

A **Capability** is a pointer to a Kernel Object which also encodes access rights to that object.

Any system operation involves **invoking** a capability (e.g. calling a functionon an object). On invocation, access rights are checked.

Advantages:

- Fine-Grained **("Object-Oriented" Access Policy)**
- Hide what the object underneath is **(Interposition)**
- Can easily **delegate** access

2.2. Kernel Objects

Kernel Objects are mainly one of the following:

- **CNodes**: Store and manage capabilities
- **Thread Control Block**: Represents a thread
- **Endpoint**: Facilitates messages between threads
- **Notification**: Used for signals
- **Virtual Address Space**: Used to construct a vspace
- **Untyped Memory**: Represents memory used by the Kernel
- **Interrupt Object**: Used to acknowledge hardware interrupts
- **Scheduling Contexts**: For MC-Systems only

2.3. Message Passing

In SeL4, messages are completely synchronous (Both partners need to be ready). There are two main types of messages:

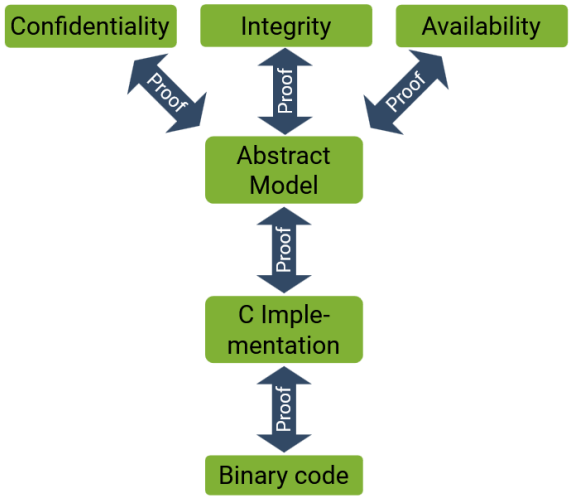
- Messages to Endpoints are destined for other threads
- Messages to other objects are processed by the Kernel

Notably, a **System Call** is a special message used to communicate with Kernel-provided services.

Each message contains a number of words, possibly a number of capabilities and a tag.

3. Verification

The security and correctness proof of SeL4 consists of three stages:



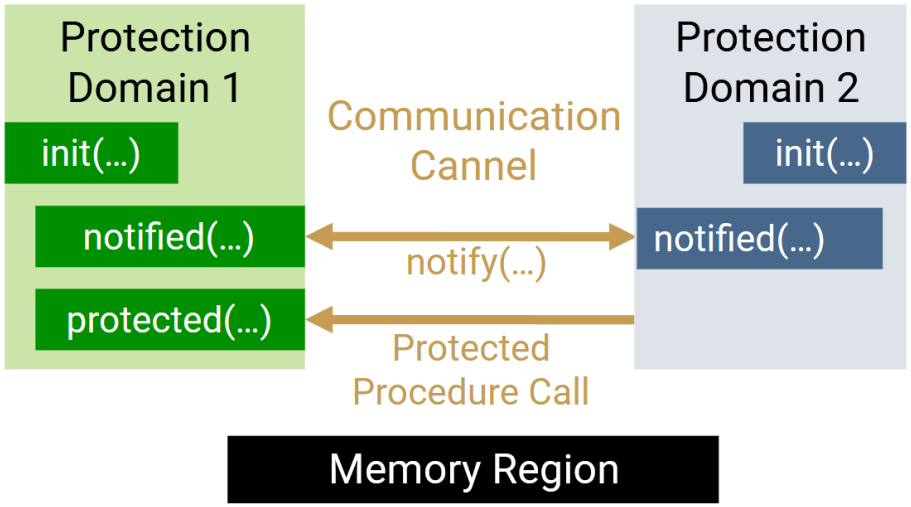
1. define an abstract model, prove it to be secure[3]
2. prove that the C code correctly implements the model
3. prove that the binary correctly implements the C code

[3] "secure" means:
confidentiality = cannot read/infer of data without read access,
integrity = cannot modify without write access,
availability = cannot hinder authorised use

4. The SeL4 Microkit

The Microkit abstracts away most of SeL4's inner workings with the goal of easing software development.

An application in the SeL4 Microkit looks like this:



The main elements are:

- Protection Domains (= Processes)
- Communication Channels between them
- Mapped Memory Regions (may be isolated or shared)

4. The SeL4 Microkit

The one downside of the Microkit is that the system's architecture needs to be static and defined beforehand.

A programmer needs to provide:

- A **System Description File** (SDF) which defines PDs, communication channels and memory regions.
- A **C Program for each PD** which has to implement the init() and notified(...) functions.

To build a complete application, one has to compile and link each C Program to the **libmicrokit** library. Then, the Microkit SDK combines all of the generated ELF files, the SDF and the SeL4 kernel into a single binary.

4.1. The System Description File

The SDF is a file written in XML. The following example showcases definitions of:

- two PDs
- a memory region used by that PD (with mapping)
- a channel between the two PDs

```
<system>

    <memory_region name="some_memory" size="0x1000" phys_addr="0x9000000" />

    <protection_domain name="pd1">
        <protection_image path="pd1.elf" />
        <map mr="some_memory" vaddr="0x4000000" perms="wr" setvar_vaddr="some_memory_vaddr"/>
    </protection_domain>

    <protection_domain name="pd2">
        ...
    </protection_domain>

    <channel>
        <end pd="pd1" id="1" />
        <end pd="pd2" id="2" />
    </channel>

</system>
```

4.2. Protection Domains

Internally, a PD includes a TCB, a virtual address-space and a capability space, but a programmer will not notice.

A programmer mainly has to define the following functions:[4]

- **init()**: Called after system boot, sets up a PD
- **notified(int ch)**: Invoked when this PD gets a notification from some channel (may be from another PD or from an IRQ)

Another thing of note:

- a programmer may declare and use pointers to memory regions used by the PD, but setting the pointers themselves is done by the SDF (**setvar_vaddr** attribute in **map** element)

[4] optional protected(...) left out here

4.3. Building for Different Platforms

To have a program be easily built for different hardware platforms, one needs to:

- change hardware-specific memory addresses and IRQ numbers (e.g. for UART)
- adapt the application code if needed

Practically, this is done by:

- Changing the .system file to a .system.template file which can be adapted by the C-Preprocessor
- Adapting the C Code with macros
- Supplying a different "platform" argument to the Microkit tool

4.3. Example Application

Idea: Two applications (a server and a client):

- server manages a "file system" with users and files
- client interacts with the user/a terminal via UART
- client sends requests to the server. Requests may be: Create_User, Create_File, Print_File, Login, Logout.
- client writes request data into a buffer[5] for the server to read
- in turn, the server writes his response into another buffer
- both PDs notify each other when writing their buffers

[5] actually three buffers. One for the request type, two for input data

4.3. Example Application - SDF Excerpt (QEMU specific)

```
<system>

    <memory_region name="uart" size="0x1000" phys_addr="0x10000000" />
    <memory_region name="request_type" size="0x1000" />
    <memory_region name="request_first_parameter" size="0x1000" />
    <memory_region name="request_second_parameter" size="0x1000" />
    <memory_region name="server_output" size="0x1000" />

    <protection_domain name="server" priority="254">
        <program_image path="server.elf" />
        <map mr="request_type" vaddr="0x4100000" perms="r" setvar_vaddr="request_type" />
        <map mr="request_first_parameter" vaddr="0x4200000" perms="r" setvar_vaddr="request_first_parameter" />
        <map mr="request_second_parameter" vaddr="0x4300000" perms="r" setvar_vaddr="request_second_parameter" />
        <map mr="server_output" vaddr="0x4400000" perms="rw" setvar_vaddr="server_output" />
    </protection_domain>

    <protection_domain name="client" priority="0">
        <program_image path="client.elf" />
        <irq id="0" irq="10"/>
        [...]
    </protection_domain>
    <channel>
        <end pd="server" id="1" />
        <end pd="client" id="2" />
    </channel>

<system>
```

4.3. Example Application - Server Excerpt

```
// This is called in the "notified" function when the channel is 1 and the request type is login
login((char*)request_first_parameter,(char*)request_second_parameter);

// Login is defined as:
void login(char* username, char* pw){

  logged_in = false;

  if(!username || username[0] == '\0'){
    copy("Empty username given!",(char*)server_output, BUFFER_LENGTH);
    return;
  }

  int index = find_user(username);
  if(index < 0){
      copy("User not found!",(char*)server_output, BUFFER_LENGTH);
      return;
  }

  if(eq(user_list[index].pw,pw)){
    copy(username, current_user, MAX_NAME_LEN);
    logged_in = true;
    copy("Logged in.",(char*)server_output, BUFFER_LENGTH);
    return;
  }

  copy("Wrong password!",(char*)server_output, BUFFER_LENGTH);
}
```

4.3. Example Application - Client Excerpt

```
// Excerpt from notified function
void notified(microkit_channel channel){
    switch(channel){
        case UART_CHANNEL_ID:

          // Handle the interrupt request and acknowledge the IRQ
          uart_handle_irq();
          microkit_irq_ack(channel);

          // Read current char and print it to the user's screen
          int chr = uart_get_char();
          uart_put_char(chr);

          // Backspace
          if (chr == 0x7f) {
            [...]
          }

          // \n reached ==> Now we have the full line and can process it depending on where we are in the current IO_STATE
          else if(chr == '\n'){
            input_buffer[buffer_index] = '\0';

            if(buffer_index > 0){
              switch(current_state){
                case READING_CHOICE:
                  process_choice();
                  break;
                case READING_FIRST_PARAM:
                  process_first_param();
                  break;
                case READING_SECOND_PARAM:
                  process_second_param();
                  break;
              }

            buffer_index = 0;

            }
          }
```

4.3. Example Application - In Use

```
SERVER: starting
CLIENT: starting
Options:
1. Create User
2. Login
3. Logout
4. Create File
5. Print File
1
Enter a username:
user
Enter a password:
pw
Server Response: User created.
Options:
1. Create User
2. Login
3. Logout
4. Create File
5. Print File
2
Enter a username:
user
Enter a password:
pw
Server Response: Logged in.
Options:
1. Create User
2. Login
3. Logout
4. Create File
5. Print File
4
Enter a filename:
file1
Enter the content of your file:
this is some example content
Server Response: File created!
Options:
1. Create User
2. Login
3. Logout
4. Create File
5. Print File
5
Enter a filename:
file1
Server Response: this is some example content
Options:
1. Create User
```

References

All information (+ the figures) is taken from:

- The SeL4 Whitepaper
- The SeL4 Manual
- The SeL4 Microkit Tutorial
- The SeL4 Microkit Manual